

JavaScript — Complete Concept Guide

In-Depth Theory + Multiple Examples, Company-Relevant Focus

Prepared for K N | Foundation for React & Web Development

Table of Contents

- 1. Fundamentals
- 2. Core Data Structures
- 3. Functions Deep Dive
- 4. Asynchronous JavaScript
- 5. ES6+ Modern Features
- 6. Object-Oriented JavaScript
- 7. Functional Programming Concepts
- 8. Browser & DOM Basics
- 9. Error Handling
- 10. Advanced / Interview Topics

1. Fundamentals

1.1 Variables: var, let, const

JavaScript has three ways to declare a variable, and each behaves differently in terms of **scope** (where it's visible) and **mutability** (whether it can be reassigned).

var — function-scoped, can be redeclared and updated, hoisted and initialized as [undefined](#).

let — block-scoped, can be updated but not redeclared in the same scope.

const — block-scoped, cannot be reassigned after declaration (but object/array contents can still change).

Example 1 — basic declaration and reassignment

```
var city = "Delhi";
city = "Mumbai"; // OK, var allows reassignment

let age = 21;
age = 22; // OK, let allows reassignment

const country = "India";
// country = "USA"; -> TypeError: Assignment to constant variable
```

Example 2 — block scope difference

```
if (true) {
  var a = 10;
  let b = 20;
}
console.log(a); // 10 - var leaks outside the block
// console.log(b); -> ReferenceError: b is not defined
```

Example 3 — const with objects/arrays

```
const user = { name: "Karan" };
user.name = "Rohit"; // allowed! we're mutating, not reassigning
// user = {} -> Error, this WOULD be reassignment
```

Gotcha: const doesn't make data immutable — it only locks the variable binding. To truly freeze an object, use [Object.freeze\(\)](#).

1.2 Data Types

JS has two categories of types:

Primitives (stored by value): string, number, boolean, null, undefined, symbol, bigint.

Non-primitives (stored by reference): objects, arrays, functions.

The distinction matters because copying a primitive copies the value, while copying an object copies the reference (both variables point to the same data).

Example 1 — typeof checks

```
console.log(typeof "hello"); // "string"
console.log(typeof 42); // "number"
console.log(typeof true); // "boolean"
console.log(typeof undefined); // "undefined"
console.log(typeof null); // "object" (a famous JS bug, kept for compatibility)
console.log(typeof {}); // "object"
console.log(typeof []); // "object" (arrays are objects)
console.log(typeof function() {}); // "function"
```

Example 2 — value vs reference copying

```
let x = 10;
let y = x;
y = 20;
console.log(x); // 10 - primitives copy by value, unaffected

let obj1 = { count: 10 };
let obj2 = obj1;
obj2.count = 20;
console.log(obj1.count); // 20 - objects copy by reference, both point to same data
```

Example 3 — null vs undefined

```
let a; // undefined - declared but not assigned
let b = null; // null - intentionally empty, assigned by developer
console.log(a == b); // true (loose equality)
console.log(a === b); // false (different types)
```

1.3 Control Flow

Control flow statements determine the order in which code runs.

Conditionals (if/else, switch) pick a path based on a condition.

Loops (for, while, for...of, for...in) repeat code.

for...of iterates over values (arrays, strings). **for...in** iterates over keys (objects).

Example 1 — if/else and switch

```
const marks = 85;
if (marks >= 90) console.log("A+");
else if (marks >= 75) console.log("A");
else console.log("B");

switch (true) {
  case marks >= 90: console.log("A+"); break;
  case marks >= 75: console.log("A"); break;
  default: console.log("B");
}
```

Example 2 — different loop types

```
const arr = [10, 20, 30];

for (let i = 0; i < arr.length; i++) {
  console.log(arr[i]); // classic index-based loop
}

for (const val of arr) {
  console.log(val); // 10 20, 30 - iterates VALUES
}
```

```

}

const obj = { a: 1, b: 2 };
for (const key in obj) {
  console.log(key); // "a", "b" - iterates KEYS
}

```

Example 3 — while and do-while

```

let count = 0;
while (count < 3) {
  console.log(count);
  count++;
} // 0, 1, 2 - condition checked BEFORE each run

let n = 5;
do {
  console.log(n);
} while (n < 3); // runs once even though condition is false

```

1.4 Functions

Functions are reusable blocks of logic. JS has three main ways to write them:

Function declaration — hoisted fully, can be called before it's defined in the file.

Function expression — assigned to a variable, NOT hoisted the same way.

Arrow function — shorter syntax, no own `this`, cannot be used as a constructor.

Example 1 — the three function forms

```

function add(a, b) {
  return a + b;
} // declaration

const subtract = function(a, b) {
  return a - b;
}; // expression

const multiply = (a, b) => a * b; // arrow function

```

Example 2 — default and rest parameters

```

function greet(name = "Guest") {
  console.log(`Hello, ${name}`);
}
greet(); // "Hello, Guest" - default used
greet("Karan"); // "Hello, Karan"

function sumAll(...nums) { // rest collects all args into an array
  return nums.reduce((a, b) => a + b, 0);
}
sumAll(1, 2, 3); // 6

```

Example 3 — hoisting difference between declaration and expression

```

sayHi(); // works! "Hi" - declarations are fully hoisted
function sayHi() { console.log("Hi"); }

// sayBye(); -> TypeError: sayBye is not a function (yet)
const sayBye = () => console.log("Bye");

```

1.5 Scope & Hoisting

Scope defines where a variable can be accessed: global scope (everywhere), function scope (inside a function), block scope (inside `{}`).

Hoisting is JS's behavior of moving declarations to the top of their scope during the compile phase, before code executes. `var` is hoisted and initialized as undefined. `let/const` are hoisted but stay uninitialized in the "Temporal Dead Zone" (TDZ) until their line runs.

Example 1 — var hoisting

```

console.log(x); // undefined (NOT an error - it's hoisted)
var x = 5;
console.log(x); // 5

```

Example 2 — let/const and the Temporal Dead Zone

```

// console.log(y); -> ReferenceError: Cannot access 'y' before initialization
let y = 5;

```

Example 3 — function scope vs block scope

```

function testScope() {
  if (true) {
    var functionScoped = "I leak out of the block";
    let blockScoped = "I stay inside";
  }
  console.log(functionScoped); // works - var is function scoped
  // console.log(blockScoped); -> ReferenceError
}

```

Why it matters for React: Using `let/const` instead of `var` avoids bugs in loops with closures (e.g. event handlers inside a `map`), which is a very common source of confusing bugs for beginners.

2. Core Data Structures

2.1 Arrays & Array Methods used daily in React

Arrays store ordered, indexed collections of values. Modern JS provides many built-in methods to transform arrays without writing manual loops. The three most important:

map() — transforms every item, returns a NEW array of the same length.

filter() — keeps items matching a condition, returns a NEW (possibly shorter) array.

reduce() — collapses the array into a single value (number, object, string, etc).

Example 1 — map, filter, reduce together

```
const nums = [1, 2, 3, 4, 5];

const doubled = nums.map(n => n * 2);
console.log(doubled); // [2, 4, 6, 8, 10]

const evens = nums.filter(n => n % 2 === 0);
console.log(evens); // [2, 4]

const total = nums.reduce((sum, n) => sum + n, 0);
console.log(total); // 15
```

Example 2 — find, some, every

```
const users = [
  { name: "Karan", age: 21 },
  { name: "Riya", age: 17 }
];

const minor = users.find(u => u.age < 18);
console.log(minor); // { name: "Riya", age: 17 } - first match

const hasMinor = users.some(u => u.age < 18);
console.log(hasMinor); // true - at least one matches

const allAdults = users.every(u => u.age >= 18);
console.log(allAdults); // false - not all match
```

Example 3 — sort, spread, and mutation trap

```
const scores = [5, 3, 9, 1];

// sort() MUTATES the original array - be careful!
const sorted = [...scores].sort((a, b) => a - b); // spread to copy first
console.log(sorted); // [1, 3, 5, 9]
console.log(scores); // [5, 3, 9, 1] - original untouched

const merged = [...scores, ...sorted]; // combine two arrays
```

Example 4 — chaining methods (very common in real code)

```
const products = [
  { name: "Pen", price: 10 },
  { name: "Book", price: 150 },
  { name: "Bag", price: 800 }
];

const expensiveNames = products
  .filter(p => p.price > 100)
  .map(p => p.name);
console.log(expensiveNames); // ["Book", "Bag"]
```

Gotcha: `sort()`, `push()`, `pop()`, `splice()`, and `reverse()` all mutate the original array. In React, mutating state directly breaks re-renders — always make a copy first.

2.2 Objects

Objects store data as key-value pairs and model real-world entities. You can access properties with dot notation (`obj.key`) or bracket notation (`obj["key"]`) — bracket notation is needed when the key is dynamic or not a valid identifier.

Example 1 — creating, accessing, updating

```
const user = { name: "Karan", age: 21, city: "Delhi" };

console.log(user.name); // "Karan" - dot notation
console.log(user["age"]); // 21 - bracket notation

user.age = 22; // update
user.email = "k@mail.com"; // add new property
delete user.city; // remove property
```

Example 2 — destructuring (extract values in one line)

```
const { name, age } = user;
console.log(name, age); // "Karan" 22

// renaming while destructuring
const { name: userName } = user;
console.log(userName); // "Karan"

// default value if property doesn't exist
const { country = "India" } = user;
console.log(country); // "India" - fallback used
```

Example 3 — Object.keys, values, entries

```
const scores = { math: 90, science: 85 };

console.log(Object.keys(scores)); // ["math", "science"]
console.log(Object.values(scores)); // [90, 85]
console.log(Object.entries(scores)); // [["math", 90], ["science", 85]]

// looping over an object using entries
for (const [subject, score] of Object.entries(scores)) {
  console.log(`${subject}: ${score}`);
}
```

Example 4 — computed property names & shorthand

```
const key = "role";
```

```
const value = "admin";

const obj = { [key]: value }; // computed key: { role: "admin" }

const x = 5, y = 10;
const point = { x, y }; // shorthand: same as { x: x, y: y }
```

2.3 Strings

Strings are immutable — any "modification" actually creates a new string. Template literals (backticks) allow embedding expressions and writing multi-line strings without concatenation.

Example 1 — template literals

```
const name = "Karan";
const age = 21;

// old way
const msg1 = "Hello, " + name + ". You are " + age + " years old.";

// modern way
const msg2 = `Hello, ${name}. You are ${age} years old.`;

// multi-line without escape characters
const multiLine = `Line 1
Line 2`;
```

Example 2 — common string methods

```
const text = "  Karan Nagar  ";

text.trim(); // "Karan Nagar" - removes whitespace
text.toLowerCase(); // "  karan nagar  "
text.includes("Nagar"); // true
text.trim().split(" "); // ["Karan", "Nagar"]
"5".padStart(3, "0"); // "005" - useful for formatting IDs
```

Example 3 — slice vs substring

```
const str = "JavaScript";
str.slice(0, 4); // "Java"
str.slice(-6); // "Script" - negative index counts from end
```

2.4 Sets & Maps

Set stores only unique values — great for removing duplicates.

Map stores key-value pairs, but unlike plain objects, keys can be ANY type (objects, functions, etc), and it preserves insertion order reliably.

Example 1 — Set for removing duplicates

```
const nums = [1, 2, 2, 3, 3, 3];
const unique = [...new Set(nums)];
console.log(unique); // [1, 2, 3]
```

Example 2 — Set methods

```
const mySet = new Set();
mySet.add("a");
mySet.add("b");
mySet.add("a"); // ignored, already exists
console.log(mySet.size); // 2
console.log(mySet.has("a")); // true
mySet.delete("a");
```

Example 3 — Map basics

```
const userRoles = new Map();
userRoles.set("karan", "admin");
userRoles.set("riya", "editor");

console.log(userRoles.get("karan")); // "admin"
console.log(userRoles.size); // 2

for (const [user, role] of userRoles) {
  console.log(`${user}: ${role}`);
}
```

Why it matters: Map is preferred over plain objects when keys are dynamic/unknown ahead of time, or when you need guaranteed order and easy size tracking.

3. Functions Deep Dive

3.1 Closures interview favorite

A closure happens when a function "remembers" the variables from the scope it was created in, even after that outer scope has finished executing. This works because JS functions carry a reference to their surrounding lexical environment, not just their own code.

Closures are the mechanism behind private variables, factory functions, and — critically for you — how React's `useState` keeps track of state between renders.

Example 1 — basic closure (counter)

```
function createCounter() {
  let count = 0; // private variable, not accessible from outside
  return function() {
    count++;
    return count;
  };
}

const counter1 = createCounter();
console.log(counter1()); // 1
console.log(counter1()); // 2 - remembers previous count

const counter2 = createCounter();
console.log(counter2()); // 1 - separate closure, own memory
```

Example 2 — closures for data privacy

```
function createBankAccount(balance) {
  return {
    deposit(amount) { balance += amount; return balance; },
    withdraw(amount) { balance -= amount; return balance; },
    getBalance() { return balance; }
  };
}

const account = createBankAccount(1000);
account.deposit(500);
console.log(account.getBalance()); // 1500
// 'balance' can't be accessed directly from outside - it's private
```

Example 3 — the classic loop + closure bug

```
// WRONG - using var
for (var i = 1; i <= 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// prints: 4, 4, 4 (var is shared across all iterations)

// CORRECT - using let
for (let i = 1; i <= 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// prints: 1, 2, 3 (let creates a new binding each iteration)
```

Why it matters for React: Every time a component re-renders, its function runs again, creating fresh closures over that render's state and props. This explains "stale closure" bugs in `useEffect` where old state values get captured.

3.2 Higher-Order Functions

A Higher-Order Function (HOF) is a function that either takes another function as an argument, returns a function, or both. This enables powerful patterns like composition, decoration, and the array methods (`map`/`filter`/`reduce`) you already use.

Example 1 — function as an argument

```
function processArray(arr, callback) {
  const result = [];
  for (const item of arr) {
    result.push(callback(item));
  }
  return result;
}

const doubled = processArray([1,2,3], n => n * 2);
console.log(doubled); // [2, 4, 6]
```

Example 2 — function returning a function

```
function multiplier(factor) {
  return (num) => num * factor;
}

const double = multiplier(2);
const triple = multiplier(3);
console.log(double(5)); // 10
console.log(triple(5)); // 15
```

Example 3 — decorator pattern with HOFs

```
function withLogging(fn) {
  return (...args) => {
    console.log("Calling with:", args);
    const result = fn(...args);
    console.log("Result:", result);
    return result;
  };
}

const loggedAdd = withLogging((a, b) => a + b);
loggedAdd(2, 3); // logs "Calling with: [2,3]" then "Result: 5"
```

3.3 The `this` Keyword

`this` refers to the object that is currently "calling" the function — but its value depends entirely on HOW the function is invoked, not where it's defined (except for arrow functions).

Rules: In a regular function called as a method, `this` is the object before the dot. In a standalone function call, `this` is undefined (strict mode) or the global object. In an arrow function, `this` is inherited from the enclosing scope — it has no `this` of its own.

Example 1 — this in a method

```
const user = {
  name: "Karan",
  greet() {
    console.log(`Hi, I'm ${this.name}`);
  }
};
user.greet(); // "Hi, I'm Karan" - this = user
```

Example 2 — losing this (a very common bug)

```
const greetFn = user.greet;
// greetFn(); -> "Hi, I'm undefined" - this is lost when detached from the object
```

Example 3 — arrow functions and this

```
const timer = {
  seconds: 0,
  start() {
    setInterval(() => {
      // arrow function inherits 'this' from start() -> timer object
      this.seconds++;
      console.log(this.seconds);
    }, 1000);
  }
};
timer.start(); // correctly increments timer.seconds
```

Gotcha: Using a regular function instead of an arrow function inside `setInterval` above would make `this` point to the global object (or undefined), breaking the counter.

3.4 call, apply, bind

These three methods let you explicitly control what `this` refers to inside a function.

call(thisArg, arg1, arg2...) — invokes the function immediately with args listed individually.

apply(thisArg, [argsArray]) — same as call, but args passed as an array.

bind(thisArg) — does NOT invoke immediately; returns a new function permanently bound to that `this`.

Example 1 — call and apply

```
function introduce(city, country) {
  console.log(`${this.name} from ${city}, ${country}`);
}
const person = { name: "Karan" };

introduce.call(person, "Delhi", "India");
// "Karan from Delhi, India"

introduce.apply(person, ["Delhi", "India"]);
// same result, args as array
```

Example 2 — bind for later use

```
function sayHi() {
  console.log(`Hi, ${this.name}`);
}
const boundFn = sayHi.bind({ name: "Riya" });
// nothing happens yet - it's just prepared

boundFn(); // "Hi, Riya" - runs now, with 'this' locked in
```

Example 3 — practical use: fixing a detached method

```
const button = {
  label: "Submit",
  handleClick() { console.log(`${this.label} clicked`); }
};

// passing button.handleClick directly loses 'this'
const fixedHandler = button.handleClick.bind(button);
fixedHandler(); // "Submit clicked" - works correctly now
```

4. Asynchronous JavaScript

4.1 The Event Loop core interview topic

JavaScript is single-threaded — it can only do one thing at a time. Async behavior is possible because of the **event loop**, which coordinates between:

Call stack — where synchronous code executes, one frame at a time.

Microtask queue — holds resolved Promises' `.then` callbacks. Runs **FIRST**, right after the current stack empties.

Macrotask (callback) queue — holds `setTimeout/setInterval` callbacks. Runs **AFTER** all microtasks are drained.

Example 1 — execution order

```
console.log("1: sync start");

setTimeout(() => console.log("2: timeout"), 0);

Promise.resolve().then(() => console.log("3: promise"));

console.log("4: sync end");

// Output order: 1, 4, 3, 2
// Sync code always runs first, then microtasks (promises), then macrotasks (timeout)
```

Example 2 — why `setTimeout(fn, 0)` doesn't run immediately

```
console.log("Start");
setTimeout(() => console.log("Timeout callback"), 0);
console.log("End");
// Output: Start, End, Timeout callback
// Even with 0ms delay, it waits for the call stack to fully clear first
```

Example 3 — multiple promises vs timeouts

```
setTimeout(() => console.log("timeout 1"), 0);
Promise.resolve("a").then(console.log);
Promise.resolve("b").then(console.log);
setTimeout(() => console.log("timeout 2"), 0);
// Output: a, b, timeout 1, timeout 2
// ALL microtasks (promises) drain before ANY macrotask runs
```

4.2 Callbacks

A callback is simply a function passed into another function to be run later — usually after some operation completes (a timer, a network request, a file read). Before Promises existed, this was the only way to handle async code, and deeply nested callbacks became known as "callback hell."

Example 1 — basic callback

```
function fetchUser(id, callback) {
  setTimeout(() => {
    callback({ id, name: "Karan" });
  }, 1000);
}

fetchUser(1, (user) => {
  console.log("Got user:", user);
});
```

Example 2 — callback hell (nested dependency)

```
fetchUser(1, (user) => {
  fetchPosts(user.id, (posts) => {
    fetchComments(posts[0].id, (comments) => {
      console.log(comments);
      // deeply nested, hard to read, hard to handle errors
    });
  });
});
```

Example 3 — error-first callback pattern (Node.js style)

```
function readFile(path, callback) {
  // simulate: callback(error, data)
  if (!path) return callback(new Error("No path given"));
  callback(null, "file contents");
}

readFile("data.txt", (err, data) => {
  if (err) return console.error(err.message);
  console.log(data);
});
```

4.3 Promises

A Promise is an object representing a value that isn't available yet but will be at some point — either successfully (**resolved**) or with an error (**rejected**). It has three states: pending, fulfilled, rejected. Promises solve callback hell by allowing chaining instead of nesting.

Example 1 — creating and consuming a Promise

```
const fetchData = new Promise((resolve, reject) => {
  setTimeout(() => {
    const success = true;
    if (success) resolve("Data loaded");
    else reject("Failed to load");
  }, 1000);
});

fetchData
  .then(result => console.log(result))
  .catch(error => console.log(error))
  .finally(() => console.log("Done, regardless of outcome"));
```

Example 2 — chaining promises (flattens nested callbacks)

```
fetchUser(1)
  .then(user => fetchPosts(user.id))
  .then(posts => fetchComments(posts[0].id))
  .then(comments => console.log(comments))
  .catch(err => console.log("Something failed:", err));
// flat, readable, single error handler for the whole chain
```

Example 3 — Promise.all vs Promise.race vs Promise.allSettled

```
const p1 = Promise.resolve(10);
const p2 = Promise.resolve(20);
const p3 = Promise.reject("error");

Promise.all([p1, p2]).then(vals => console.log(vals)); // [10, 20] - waits for ALL

Promise.race([p1, p2]).then(val => console.log(val)); // 10 - whichever resolves FIRST

Promise.allSettled([p1, p3]).then(results => console.log(results));
// [{status:"fulfilled",value:10}, {status:"rejected",reason:"error"}]
// never short-circuits, gives you the outcome of EVERY promise
```

4.4 async/await

daily use in React

async/await is syntactic sugar built on top of Promises — it lets you write asynchronous code that reads top-to-bottom like synchronous code, without chains of `.then()`. A function marked **async** always returns a Promise. **await** pauses execution of that function (not the whole program) until the Promise resolves.

Example 1 — basic async/await with try/catch

```
async function getUser() {
  try {
    const res = await fetch("https://api.example.com/user");
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.log("Error fetching user:", err.message);
  }
}
getUser();
```

Example 2 — sequential vs parallel awaits

```
// SLOW - sequential (each waits for the previous to finish)
async function sequential() {
  const user = await fetchUser();
  const posts = await fetchPosts(); // waits unnecessarily
}

// FAST - parallel (both start at the same time)
async function parallel() {
  const [user, posts] = await Promise.all([
    fetchUser(),
    fetchPosts()
  ]);
}
```

Example 3 — using async/await inside a React-style effect

```
function useUserData(userId) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    async function load() {
      const res = await fetch(`/api/users/${userId}`);
      const data = await res.json();
      setUser(data);
    }
    load();
  }, [userId]);

  return user;
}
// note: useEffect's callback itself can't be async directly,
// so we define and call an inner async function
```

Gotcha: You cannot make the function passed directly to `useEffect` async — React expects it to return either nothing or a cleanup function, not a Promise. Always wrap async logic in an inner function.

5. ES6+ Modern Features

5.1 Destructuring used constantly in React

Destructuring lets you unpack values from arrays or properties from objects into individual variables in a single, readable line — instead of accessing each one separately. This is the standard way React components read their props.

Example 1 — array destructuring

```
const coords = [10, 20, 30];
const [x, y, z] = coords;
console.log(x, y, z); // 10 20 30

// skipping values
const [first, , third] = coords;
console.log(first, third); // 10 30

// swapping variables without a temp variable
let a = 1, b = 2;
[a, b] = [b, a];
console.log(a, b); // 2 1
```

Example 2 — object destructuring with renaming & defaults

```
const user = { name: "Karan", age: 21 };

const { name: fullName, age, country = "India" } = user;
console.log(fullName, age, country); // "Karan" 21 "India"
```

Example 3 — destructuring in function parameters (React props pattern)

```
function ProductCard({ title, price, discount = 0 }) {
  const finalPrice = price - discount;
  return `${title}: ₹${finalPrice}`;
}
ProductCard({ title: "Pen", price: 10 });
// destructuring right in the parameter list - very common in React
```

Example 4 — nested destructuring

```
const response = {
  data: { user: { name: "Karan", address: { city: "Delhi" } } }
};
const { data: { user: { address: { city } } } } = response;
console.log(city); // "Delhi"
```

5.2 Spread & Rest Operators

Both use the same `...` syntax but do opposite jobs.

Spread expands an array/object into individual elements — used to copy or merge data.

Rest gathers multiple individual values into a single array — used in function parameters or destructuring.

Spread is critical for React because state must be updated immutably — you always create a new copy rather than mutating the original.

Example 1 — spread with arrays

```
const arr1 = [1, 2];
const arr2 = [3, 4];

const combined = [...arr1, ...arr2]; // [1, 2, 3, 4]
const copy = [...arr1]; // [1, 2] - independent copy
const withNew = [...arr1, 99]; // [1, 2, 99]
```

Example 2 — spread with objects (React state updates)

```
const user = { name: "Karan", age: 21 };

const updatedUser = { ...user, age: 22 };
console.log(updatedUser); // { name: "Karan", age: 22 } - new object, original untouched

// typical React state update pattern:
// setUser(prev => ({ ...prev, age: prev.age + 1 }));
```

Example 3 — rest parameters in functions

```
function sum(first, ...rest) {
  console.log(first); // first argument alone
  console.log(rest); // remaining arguments as an array
  return rest.reduce((total, n) => total + n, first);
}
sum(1, 2, 3, 4); // first=1, rest=[2,3,4], returns 10
```

Example 4 — rest in destructuring

```
const { name, ...otherDetails } = { name: "Karan", age: 21, city: "Delhi" };
console.log(name); // "Karan"
console.log(otherDetails); // { age: 21, city: "Delhi" }
```

Gotcha: Spread only creates a SHALLOW copy. If an object contains nested objects/arrays, those inner references are still shared. For deeply nested state, you need to spread at every level or use a helper like `structuredClone()`.

5.3 Optional Chaining & Nullish Coalescing

Optional chaining (`?.`) safely accesses deeply nested properties — if any part in the chain is null/undefined, it short-circuits to undefined instead of throwing an error.

Nullish coalescing (`??`) provides a fallback value, but ONLY when the left side is null or undefined — unlike `||`, which also treats 0, "", and false as "falsy" and would incorrectly fall back for those.

Example 1 — optional chaining avoiding crashes

```
const user = { name: "Karan" }; // no 'address' property

// console.log(user.address.city); -> TypeError: Cannot read property 'city' of undefined
console.log(user.address?.city); // undefined - no crash
console.log(user.address?.city ?? "Unknown"); // "Unknown"
```

Example 2 — optional chaining with function calls

```
const obj = { greet: null };
obj.greet?(); // does nothing, no error - checks if greet exists before calling
```

Example 3 — nullish coalescing vs logical OR (important distinction)

```
const quantity = 0;

console.log(quantity || 10); // 10 - WRONG, treats 0 as falsy
console.log(quantity ?? 10); // 0 - CORRECT, 0 is a valid value, not null/undefined

const username = "";
console.log(username || "Guest"); // "Guest" - WRONG if empty string was intentional
console.log(username ?? "Guest"); // "" - CORRECT, empty string is kept
```

5.4 Modules (import/export)

Modules let you split code across multiple files and share functionality between them. Every file is its own scope by default — you explicitly **export** what should be usable elsewhere, and **import** it where needed. There are two export styles: **named exports** (multiple per file) and **default export** (one per file).

Example 1 — named exports

```
// mathUtils.js
export const add = (a, b) => a + b;
export const subtract = (a, b) => a - b;

// app.js
import { add, subtract } from "./mathUtils";
console.log(add(2, 3)); // 5
```

Example 2 — default export

```
// Button.js
function Button() { return "I'm a button"; }
export default Button;

// app.js — default exports can be imported with any name
import MyButton from "./Button";
```

Example 3 — mixing default and named exports (common in React files)

```
// UserCard.js
export const DEFAULT_AVATAR = "/avatar.png";
export default function UserCard() { ... }

// app.js
import UserCard, { DEFAULT_AVATAR } from "./UserCard";
```

5.5 Classes

Classes are syntactic sugar over JavaScript's prototype system — they give a cleaner, more familiar syntax (similar to other OOP languages) for creating blueprints of objects with shared behavior.

Example 1 — basic class with constructor

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  greet() {
    return `Hi, I'm ${this.name}, ${this.age} years old`;
  }
}

const p = new Person("Karan", 21);
console.log(p.greet()); // "Hi, I'm Karan, 21 years old"
```

Example 2 — static methods and getters

```
class MathHelper {
  static square(n) { return n * n; } // called on the class itself, not an instance
}

console.log(MathHelper.square(5)); // 25 - no need to instantiate

class Circle {
  constructor(radius) { this.radius = radius; }
  get area() { return Math.PI * this.radius ** 2; }
}

const c = new Circle(5);
console.log(c.area); // accessed like a property, not called like a method
```

6. Object-Oriented JavaScript

6.1 Prototypes & Prototypal Inheritance

Every object in JS has an internal, hidden link to another object called its **prototype**. When you access a property that doesn't exist directly on an object, JS automatically looks up the prototype chain. This is the mechanism underneath classes, array methods, and virtually everything in the language.

Example 1 — prototype-based sharing (before classes existed)

```
function Animal(name) {
  this.name = name;
}
Animal.prototype.speak = function() {
  console.log(`${this.name} makes a sound`);
};

const dog = new Animal("Dog");
const cat = new Animal("Cat");
dog.speak(); // "Dog makes a sound" - shared method, not duplicated per instance
cat.speak(); // "Cat makes a sound"
```

Example 2 — checking the prototype chain

```
console.log(dog.__proto__ === Animal.prototype); // true
console.log(dog instanceof Animal);             // true
```

Example 3 — why prototypes matter for memory efficiency

```
// BAD - every instance gets its own copy of the function (wastes memory)
function Car(model) {
  this.model = model;
  this.drive = function() { console.log("Driving"); };
}

// GOOD - one shared function on the prototype, used by all instances
function Car2(model) { this.model = model; }
Car2.prototype.drive = function() { console.log("Driving"); };
```

6.2 Classes & Inheritance (extends)

extends allows one class to inherit properties and methods from another (the parent/base class). **super()** must be called inside the child's constructor before using **this** — it runs the parent's constructor logic first. This models "is-a" relationships (a Student IS A Person).

Example 1 — basic inheritance

```
class Person {
  constructor(name) { this.name = name; }
  greet() { return `Hi, I'm ${this.name}`; }
}

class Student extends Person {
  constructor(name, course) {
    super(name); // calls Person's constructor
    this.course = course;
  }
  study() { return `${this.name} is studying ${this.course}`; }
}

const s = new Student("Karan", "CS");
console.log(s.greet()); // "Hi, I'm Karan" - inherited method
console.log(s.study()); // "Karan is studying CS" - own method
```

Example 2 — overriding a parent method

```
class Animal {
  speak() { return "Some generic sound"; }
}
class Dog extends Animal {
  speak() { return "Woof!"; } // overrides parent's speak()
}
console.log(new Dog().speak()); // "Woof!"
```

Example 3 — calling the parent's method with super

```
class Shape {
  describe() { return "I am a shape"; }
}
class Circle extends Shape {
  describe() {
    return super.describe() + " and specifically a circle";
  }
}
console.log(new Circle().describe()); // "I am a shape and specifically a circle"
```

7. Functional Programming Concepts

7.1 Pure Functions

A pure function always produces the same output for the same input, and has NO side effects — it doesn't modify any external state, doesn't touch the DOM, doesn't log, doesn't make network calls. Pure functions are predictable, easy to test, and easy to reason about. React relies heavily on this idea: your components should ideally render the same UI given the same props/state.

Example 1 — pure vs impure function

```
// Pure - depends only on inputs, no external effects
function add(a, b) { return a + b; }
add(2, 3); // always 5, no matter when/how often called

// Impure - depends on and modifies external state
let total = 0;
function addToTotal(n) {
  total += n; // side effect: modifies variable outside its scope
  return total;
}
```

Example 2 — pure function with arrays (no mutation)

```
// Impure - mutates the original array
function addItem(arr, item) {
  arr.push(item); // modifies the array passed in
  return arr;
}

// Pure - returns a new array, original untouched
function addItemPure(arr, item) {
  return [...arr, item];
}
```

Example 3 — why purity matters for testing

```
// Easy to test - same input always gives same output
function calculateTax(price, rate) {
  return price * rate;
}

// test: expect(calculateTax(100, 0.1)).toBe(10) - reliable, no setup needed
```

7.2 Immutability critical for React state

Immutability means never changing data directly — instead you create a new copy with the changes applied. React uses this to detect whether state has changed (via reference comparison), which determines whether a component re-renders. Mutating state directly can cause React to silently miss updates.

Example 1 — mutating vs creating a new array

```
const arr = [1, 2, 3];

// WRONG - mutation, same reference, React won't detect a change
arr.push(4);

// RIGHT - new array, new reference
const newArr = [...arr, 4];
```

Example 2 — updating nested objects immutably

```
const state = { user: { name: "Karan", age: 21 } };

// WRONG
state.user.age = 22;

// RIGHT - spread at every nested level being changed
const newState = {
  ...state,
  user: { ...state.user, age: 22 }
};
```

Example 3 — immutable update inside React's setState pattern

```
// removing an item from an array immutably
const [todos, setTodos] = useState([
  {id:1, text:"Learn JS"},
  {id:2, text:"Learn React"}
]);

function removeTodo(id) {
  setTodos(prev => prev.filter(todo => todo.id !== id));
  // filter returns a NEW array - React detects the change and re-renders
}
```

7.3 Function Composition & Currying

Composition combines small, single-purpose functions into a bigger operation by feeding the output of one into the next.

Currying transforms a function that takes multiple arguments into a sequence of functions, each taking one argument. It's useful for creating specialized, reusable versions of a function.

Example 1 — composing two functions manually

```
const double = n => n * 2;
const addTen = n => n + 10;

const compose = (f, g) => (x) => f(g(x));
const doubleThenAddTen = compose(addTen, double);
console.log(doubleThenAddTen(5)); // double(5)=10, then addTen(10)=20
```

Example 2 — currying basics

```
// normal function
function multiply(a, b) { return a * b; }
multiply(2, 5); // 10

// curried version
const curriedMultiply = a => b => a * b;
const double = curriedMultiply(2); // partially applied
console.log(double(5)); // 10
console.log(double(10)); // 20 - reused with a new value
```

Example 3 — practical currying (creating specialized functions)

```
const discount = rate => price => price - (price * rate);

const tenPercentOff = discount(0.10);
const fiftyPercentOff = discount(0.50);

console.log(tenPercentOff(100)); // 90
console.log(fiftyPercentOff(100)); // 50
```

8. Browser & DOM Basics

8.1 DOM Manipulation

The DOM (Document Object Model) is the browser's live, tree-shaped representation of your HTML. JavaScript can select elements, read/change their content, and respond to user interaction. React actually abstracts this away using a "virtual DOM," but understanding raw DOM manipulation helps you understand what React is doing under the hood.

Example 1 — selecting and modifying elements

```
const heading = document.querySelector("h1");
heading.textContent = "Updated Title";
heading.style.color = "blue";

const allButtons = document.querySelectorAll("button"); // returns a NodeList
allButtons.forEach(btn => btn.style.padding = "8px");
```

Example 2 — creating and appending elements

```
const newDiv = document.createElement("div");
newDiv.textContent = "I'm new here";
newDiv.classList.add("card");
document.body.appendChild(newDiv);
```

Example 3 — handling events

```
const btn = document.querySelector("#submit-btn");
btn.addEventListener("click", (event) => {
  event.preventDefault(); // stops default browser behavior (e.g. form submit)
  console.log("Button clicked");
});
```

8.2 Event Bubbling & Delegation

When an event fires on an element, it "bubbles up" through all its parent elements too. **Event delegation** exploits this: instead of attaching a listener to every child element, you attach ONE listener to a shared parent and check which child triggered it. This is more memory-efficient, especially for dynamic lists.

Example 1 — demonstrating bubbling

```
// clicking the button also triggers the parent div's listener
document.querySelector("div").addEventListener("click", () => {
  console.log("Div clicked (bubbled from child)");
});
document.querySelector("button").addEventListener("click", () => {
  console.log("Button clicked");
});
// clicking button logs: "Button clicked" THEN "Div clicked (bubbled from child)"
```

Example 2 — event delegation on a list

```
// instead of adding a listener to every
•, add ONE to the

document.querySelector("ul").addEventListener("click", (e) => {
  if (e.target.tagName === "LI") {
    console.log("Clicked item:", e.target.textContent);
  }
});
// works even for
• elements added to the list LATER
```

Example 3 — stopping propagation when needed

```
childElement.addEventListener("click", (e) => {
  e.stopPropagation(); // prevents the event from bubbling to parents
  console.log("Only this handler runs");
});
```

8.3 localStorage / sessionStorage

Both provide simple key-value storage in the browser, limited to strings.

localStorage persists even after closing the browser/tab.

sessionStorage clears when the tab is closed.

Objects must be serialized with `JSON.stringify()` before storing and parsed back with `JSON.parse()` when read.

Example 1 — basic usage

```
localStorage.setItem("token", "abc123");
const token = localStorage.getItem("token");
console.log(token); // "abc123"

localStorage.removeItem("token");
localStorage.clear(); // removes everything
```

Example 2 — storing and retrieving objects

```
const user = { name: "Karan", age: 21 };

localStorage.setItem("user", JSON.stringify(user));

const savedUser = JSON.parse(localStorage.getItem("user"));
console.log(savedUser.name); // "Karan"
```

Example 3 — sessionStorage for temporary data

```
sessionStorage.setItem("formDraft", JSON.stringify({ title: "Untitled" }));
// cleared automatically when the tab/browser closes - good for drafts, temp state
```

9. Error Handling

9.1 try/catch/finally

`try` wraps code that might throw an error. `catch` runs if an error occurs, receiving the error object. `finally` always runs afterward, regardless of whether an error occurred — commonly used for cleanup (closing connections, hiding loaders).

Example 1 — basic error catching

```
try {
  const data = JSON.parse("{invalid json}");
} catch (error) {
  console.log("Parsing failed:", error.message);
}
```

Example 2 — finally for cleanup

```
function loadData() {
  showLoader();
  try {
    riskyOperation();
  } catch (err) {
    console.log("Failed:", err.message);
  } finally {
    hideLoader(); // always runs - success OR failure
  }
}
```

Example 3 — try/catch with async/await

```
async function fetchUserSafely(id) {
  try {
    const res = await fetch(`/api/users/${id}`);
    if (!res.ok) throw new Error(`Status ${res.status}`);
    return await res.json();
  } catch (err) {
    console.log("Could not fetch user:", err.message);
    return null;
  }
}
```

9.2 Custom Errors

You can extend the built-in `Error` class to create your own specific error types. This makes debugging easier because you can check `error.name` or use `instanceof` to handle different error types differently.

Example 1 — creating a custom error class

```
class ValidationError extends Error {
  constructor(message) {
    super(message);
    this.name = "ValidationError";
  }
}

function validateAge(age) {
  if (age < 0) {
    throw new ValidationError("Age cannot be negative");
  }
  return age;
}
```

Example 2 — catching and distinguishing custom errors

```
try {
  validateAge(-5);
} catch (err) {
  if (err instanceof ValidationError) {
    console.log("Validation issue:", err.message);
  } else {
    console.log("Unexpected error:", err.message);
  }
}
```

Example 3 — multiple custom error types

```
class NetworkError extends Error {
  constructor(message) { super(message); this.name = "NetworkError"; }
}

class AuthError extends Error {
  constructor(message) { super(message); this.name = "AuthError"; }
}

// lets calling code respond differently to different failure types
```

10. Advanced / Interview Topics

10.1 Debounce common interview + real use

Debouncing delays running a function until a certain amount of time has passed WITHOUT it being called again. Every new call resets the timer. This is used for search inputs, so an API call fires only after the user stops typing — not on every keystroke.

Example 1 — building a debounce function from scratch

```
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer); // cancel the previous pending call
    timer = setTimeout(() => fn(...args), delay);
  };
}
```

Example 2 — using debounce on a search input

```
function fetchResults(query) {
  console.log("Searching for:", query);
}
const debouncedSearch = debounce(fetchResults, 300);

// simulating rapid typing - only the LAST call after 300ms of silence runs
debouncedSearch("h");
debouncedSearch("he");
debouncedSearch("hel");
debouncedSearch("hell");
debouncedSearch("hello");
// only "hello" actually triggers fetchResults, after typing stops
```

Example 3 — debounce in a React input handler

```
function SearchBox() {
  const handleChange = debounce((e) => {
    fetchResults(e.target.value);
  }, 300);

  return `
    <input type="text" value="" onChange={handleChange} />
  `;
}
```

10.2 Throttle

Throttling ensures a function runs at most ONCE within a given time interval, no matter how many times it's triggered. Unlike debounce (which waits for silence), throttle guarantees regular execution — useful for scroll, resize, or mouse-move handlers where you want updates, just not on EVERY single event.

Example 1 — building a throttle function

```
function throttle(fn, limit) {
  let waiting = false;
  return function(...args) {
    if (!waiting) {
      fn(...args);
      waiting = true;
      setTimeout(() => { waiting = false; }, limit);
    }
  };
}
```

Example 2 — throttle on a scroll event

```
function handleScroll() {
  console.log("Scroll position:", window.scrollY);
}
const throttledScroll = throttle(handleScroll, 200);
window.addEventListener("scroll", throttledScroll);
// fires at most once every 200ms, no matter how fast the user scrolls
```

Example 3 — debounce vs throttle side by side

```
// DEBOUNCE: waits for a pause, then runs once
// good for: search-as-you-type, form validation on typing

// THROTTLE: runs at a regular, limited rate no matter what
// good for: scroll position tracking, drag events, button spam prevention
```

10.3 Memoization

Memoization caches the result of an expensive function call, so if it's called again with the SAME input, the cached result is returned instantly instead of recomputing. This is the underlying idea behind React's [useMemo](#) and [useCallback](#) hooks.

Example 1 — building a memoize wrapper

```
function memoize(fn) {
  const cache = {};
  return function(n) {
    if (n in cache) {
      console.log("From cache");
      return cache[n];
    }
    const result = fn(n);
    cache[n] = result;
    return result;
  };
}
```

Example 2 — memoizing an expensive calculation

```
function slowSquare(n) {
  for (let i = 0; i < 1000000000; i++) {} // simulate heavy work
  return n * n;
}
const fastSquare = memoize(slowSquare);

fastSquare(5); // slow - computes and caches
fastSquare(5); // instant "From cache"
```

Example 3 — the React equivalent (useMemo)

```
function ProductList({ products, filter }) {
  const filtered = useMemo(() => {
    return products.filter(p => p.category === filter);
  }, [products, filter]);
  // only recalculates when 'products' or 'filter' actually change
  return filtered;
}
```

10.4 Design Patterns (Module, Observer)

Module pattern uses closures to create private state and expose only what's needed — an early form of encapsulation in JS. **Observer pattern** lets objects ("subscribers") register interest in events and get notified when they happen. This is the conceptual root of React's state-update-triggers-re-render model, and of pub/sub systems in general.

Example 1 — module pattern

```
const CounterModule = (function() {
  let count = 0; // private, inaccessible from outside
  return {
    increment() { return ++count; },
    reset() { count = 0; }
  };
})();
CounterModule.increment(); // 1
CounterModule.increment(); // 2
// CounterModule.count is undefined - fully private
```

Example 2 — observer pattern (event emitter)

```
class EventEmitter {
  listeners = {};

  on(event, callback) {
    if (!this.listeners[event]) this.listeners[event] = [];
    this.listeners[event].push(callback);
  }

  emit(event, data) {
    (this.listeners[event] || []).forEach(cb => cb(data));
  }
}

const emitter = new EventEmitter();
emitter.on("userLoggedIn", (user) => console.log(`Welcome, ${user}`));
emitter.emit("userLoggedIn", "Karan"); // "Welcome, Karan"
```

Example 3 — why this matters for React mental models

```
// React state works conceptually like an observer pattern:
// 1. Component "subscribes" to a piece of state via useState
// 2. When setState is called, React "emits" a re-render
// 3. The component function re-runs and reflects the new value
// You don't manually wire this - React's internals handle the subscription
```